CS 3006
Parallel and Distributed Computing
Lecture 16

Danyal Farhat

FAST School of Computing

NUCES Lahore

Introduction to Distributed Systems & Types of Distributed System Architecture

Lecture's Agenda

- **Classic Computing - Pros and Cons**
- **Distributed System**
- **Reasons for Distributed Systems**
- **Examples of Distributed Systems**
- **Search Engine**
- **Advantages of Distributed Systems**
- **Trends in Distributed Systems**
  - **Pervasive Network Technology**
  - **Mobile and Ubiquitous Computing**
  - **Spontaneous Interoperation**

Lecture's Agenda (Cont.)

- **Trends in Distributed Systems (Cont.)**
  - **Distributed Computing as a Utility**
  - **Focus on resource sharing**
- **Forms of Distributed Computing**
  - **Cluster Computing**
  - **Grid Computing**
  - **Cloud Computing**
- **Message Passing Paradigm**
- **Communication Model**
- **Parameters for Measuring Arrival Latency**

Lecture's Agenda (Cont.)

- **Architectures**
  - **Client-Server (C/S) Architecture**
  - **Peer-to-Peer (P2P) Architecture**
- **Client/Server Paradigm**
- **Synchronous Message Passing**
- **Asynchronous Message Passing**
- **Peer to Peer System Architecture**
- **Distributed Systems: Issues**
- **Summary**

Classic Computing

- **Advantages:**
  - Ease of programming (shared memory)
  - Robustness
  - Good chance of optimization
- **Disadvantages:**
  - Data protection from illegal operations
  - Low performance
  - Single CPU, sequential computation
    - ✓ Mitigated with multi-CPU systems and concurrent programming (thread, processes)
  - Requires physical access to the system (for usage)
    - ✓ Mitigated with modem / network connections

Distributed System

- **Networks of computers are everywhere!**
  - **Mobile phone networks**
  - **Campus networks**
  - **In-car networks**
  - **On board networks in planes and trains**

- **A distributed System is:**

A collection of independent computers that appears to its users as a single coherent system

Distributed System (Cont.)

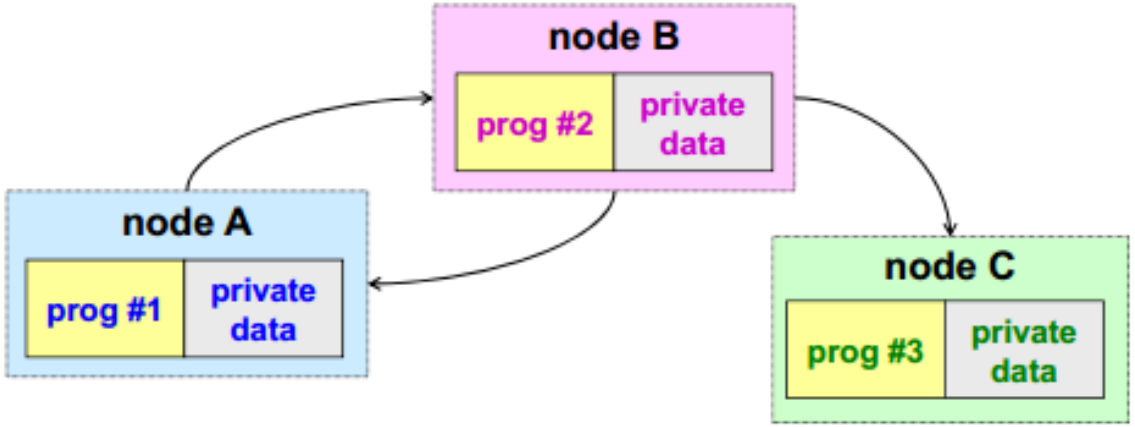
- **"A system in which hardware or software components located at networked computers communicate and coordinate their actions only by message passing." [Coulouris]**
- **"A distributed system is a collection of independent computers that appear to the users of the system as a single computer."**

  [Tanenbaum]
- **"A distributed system is a system designed to support the development of applications and services which can exploit a physical architecture consisting of multiple autonomous processing elements that do not share primary memory but cooperate by sending asynchronous messages over a communications network"**

  [Blair and Stefani, 1998]

Distributed System (Cont.)

- Each system has its local data (private)
- Multiple address spaces
- Concurrent computation over different CPUs
- Multiple computation flows



Reasons for Distributed Systems

Functional Separation:

- **Existence of computers with different capabilities and purposes:**
  - **Clients and Servers**
  - **Data collection and data processing**



Inherent Distribution:

- **Information**
  - **Different information is created and maintained by different people (e.g. Web pages)**
- **People**
  - **Computer supported collaborative work (virtual teams, engineering, virtual surgery)**
- **Retail store and inventory systems for supermarket chains (e.g., Coles, Woolworths)**

Reasons for Distributed Systems (Cont.)

Power Imbalance and Load Variation:

- **Distribute computational load among different computers**

Reliability:

- **Long term preservation and data backup (replication) at different locations**



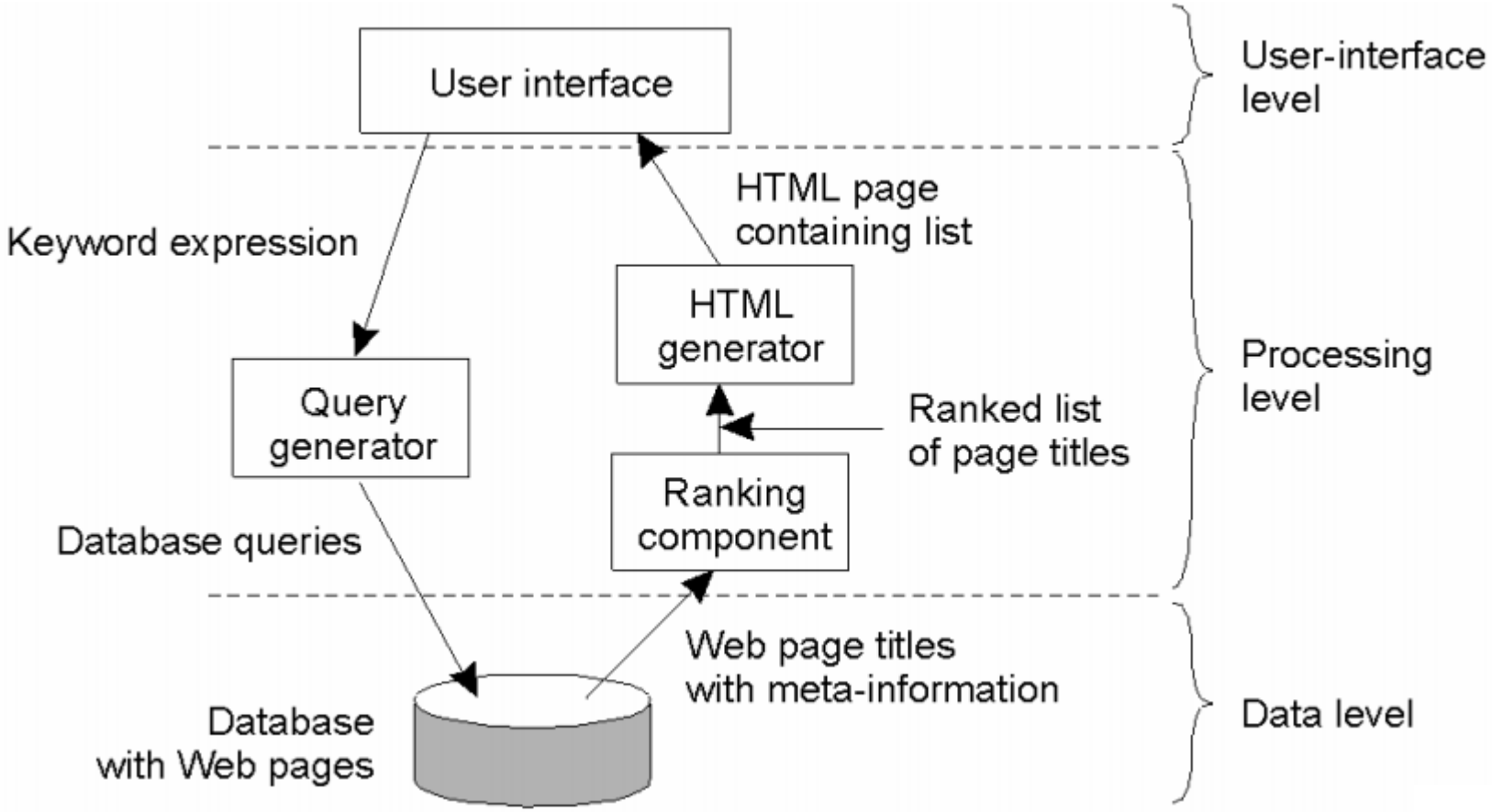
Economies:

- **Sharing a printer by many users and reduce the cost of ownership**
- **Building a supercomputer out of a network of computers**

Examples of Distributed Systems

- **Internet: network of networks**
  - **Global access to everyone**
  - **Huge size**
  - **No single authority**
- **Intranets**
  - **Protected access**
  - **Single authority**
- **E-learning**
- **Online gaming**
- **Mobile ad-hoc networks**

Search Engine – An Example of Distributed Systems



- **http://www.ece.rutgers.edu/~parashar**

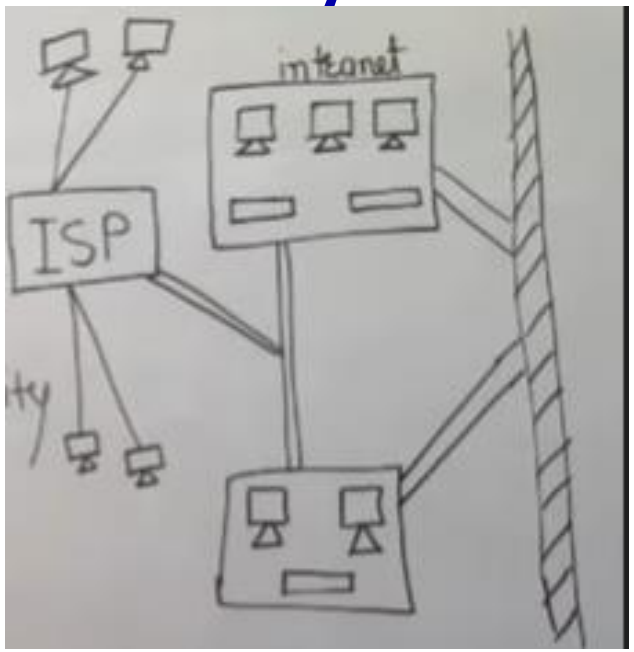
Advantages of Distributed Systems

- **High performance**
  - **Several CPUs**
- **Good scalability**
  - **Increasing the number of CPUs is easier then increasing the performance of a single CPU**
- **Data protected from illegal operations**
  - **Disjoint memory spaces, accessible only by their respective programs**
- **Network access**
  - **User physical presence not required**

Trends in Distributed Systems

Pervasive Network Technology:

- **Modern internet is a vast interconnected collection of computer networks of many different types**
- **Range of types increasing all the time**
- **Devices can be connected at any time and at any place**



Trends in Distributed Systems (Cont.)

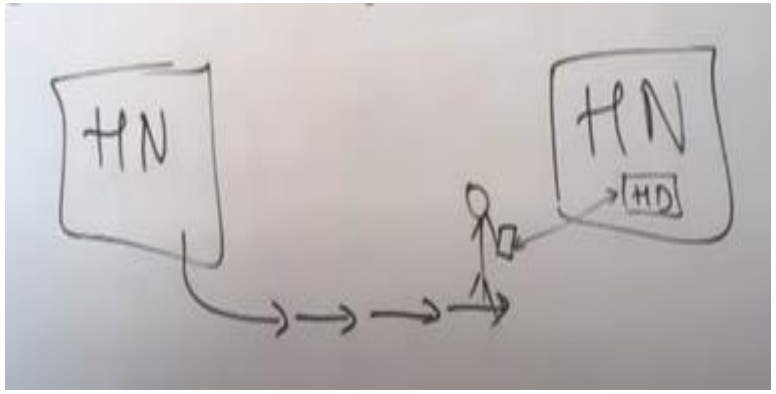
Mobile and Ubiquitous Computing:

- **Integration of small and portable computing devices into distributed systems**
- **Laptops, PDAs, smart watches, smart embedded devices in appliances**
- **Location-aware or context-aware computing**
- **Presence of computers everywhere only becomes useful when they can communicate with one another**
- **It is more desirable for users to control their washing machine or their entertainment system from their phone or a 'universal remote control' device in the home**

Trends in Distributed Systems (Cont.)

Spontaneous Interoperation:

- **Associations between devices are routinely created and destroyed**
- **Challenges: should be fast and convenient**



Trends in Distributed Systems (Cont.)

Distributed Computing as a Utility:

- **Resources are provided by service suppliers and effectively rented rather than owned by the end user**
  - **Cloud Computing**

Focus on resource sharing

- **We routinely share hardware resources such as printers, files, and resources with more specific functionality such as search engines**
  - **Client/Server computing**
  - **Peer to Peer computing**

Forms of Distributed Computing

Cluster Computing:

- **"A computer cluster is a group of loosely coupled computers that work together closely so that in many respects it can be viewed as though it were a single computer. Clusters are commonly connected through fast local area networks."**

Forms of Distributed Computing (Cont.)

Grid Computing:

- **"Grid is an infrastructure that involves the integrated and collaborative use of computers, networks, databases and scientific instruments owned and managed by multiple organizations."**
- **Grid Computing is biased towards the solution of scientific problems**

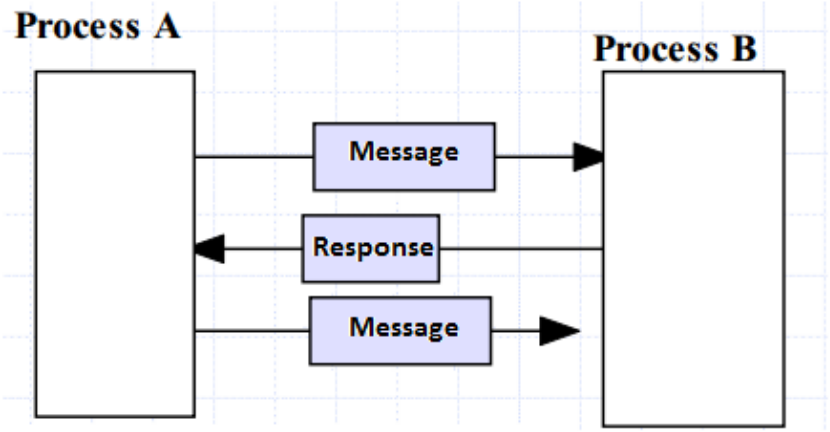
Forms of Distributed Computing (Cont.)

Cloud Computing:

- **"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction." (NIST, USA)**
- **It is also defined as a set of Internet-based application, storage and computing services sufficient to support most users' needs, thus enabling them to largely or totally dispense with local data storage and application software**

Message Passing Paradigm

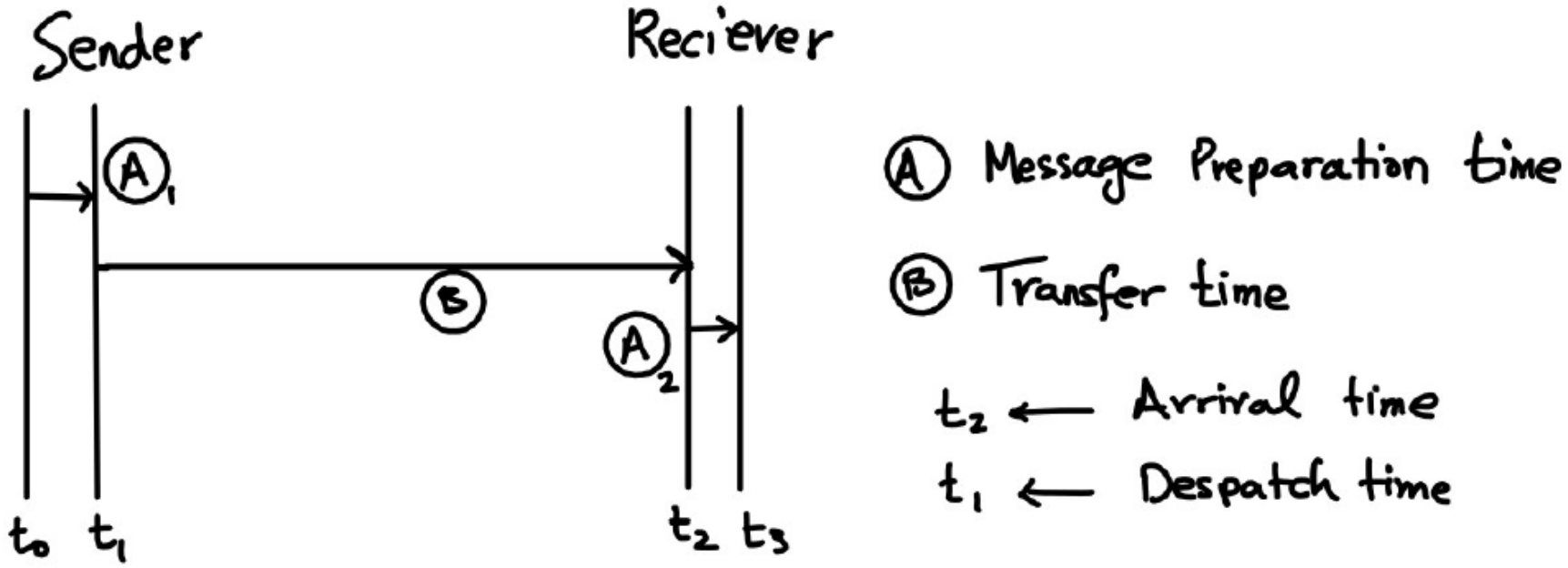
- **Most fundamental paradigm for distributed applications**



- **A process sends a message representing a request**
- **The message is delivered to a receiver, which processes the request, and sends a message in response**
- **In turn, the reply may trigger a further request, which leads to a subsequent reply, and so forth**

Communication Model

- **Message preparation & Message transfer time**



- Arrival latency = transfer time = t2 – t1

Parameters for Measuring Arrival Latency

Bandwidth : Data Volume (r) sent / unit time

e.g 400 mbits / second => 1 mbit transfer = $\frac{1s}{400}$

1 unit transfer = 1 / r

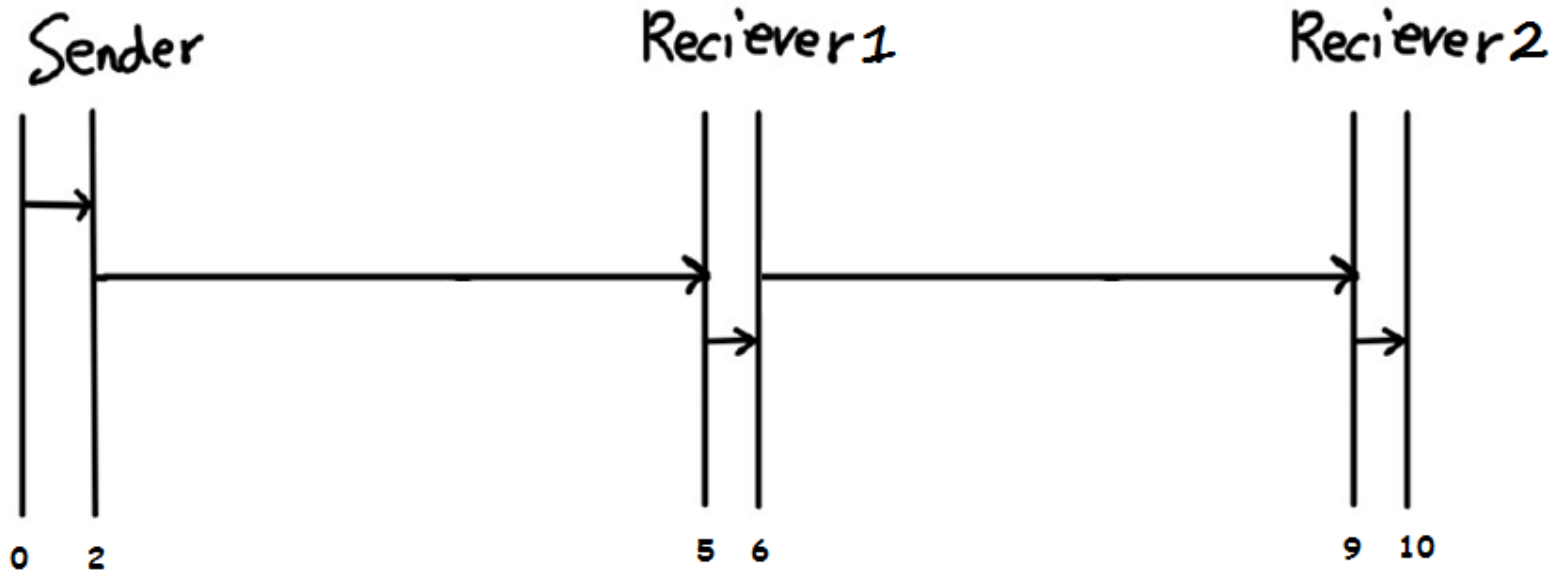
- T_w = 1 unit transfer time = 1/r
- T_s = preparation time = $t_1 - t_0$
- T_h = Arrival latency = $t_2 - t_1$
- T_r = Receiver handling time = $t_3 - t_2$
- Size of message = m
- No. of hops/nodes, the message is to be sent across = n

Parameters for Measuring Arrival Latency (Cont.)

- **Total time for sending a message of size m across n hops:**

$$t = t_s + (t_w\, m + t_h + t_r)\, n$$

- **Example:**



Example

- **bandwidth = 2 mbits/s**
- **m = 100 mbits**
- **n = 2**

- **T_w = 1 unit transfer time = 1/2**
- **$T_s = t_1 - t_0 = 2 - 0 = 2$**
- **$T_h = t_2 - t_1 = 5 - 2 = 3$**
- **$T_r = t_3 - t_2 = 6 - 5 = 1$**

- **$t = t_s + (t_w\, m + t_h + t_r)\, n = 2 + (½\ \ 100 + 3 + 1)\, 2 \quad = 110$ s**

Architectures

Client-Server (C/S) Architecture:

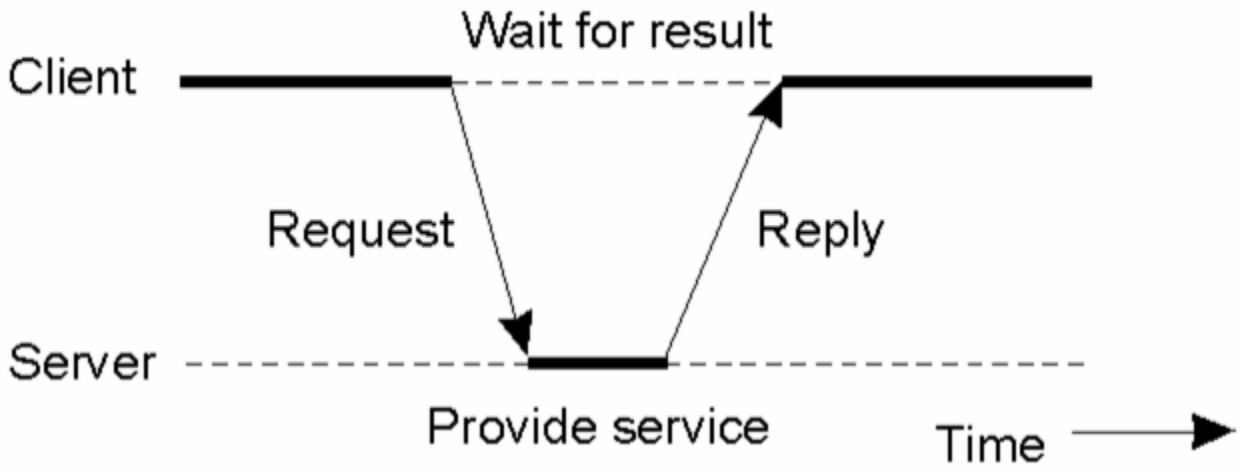
- **Asymmetric architecture**
- **Server position is determined a priori**

Peer-to-Peer (P2P) Architecture:

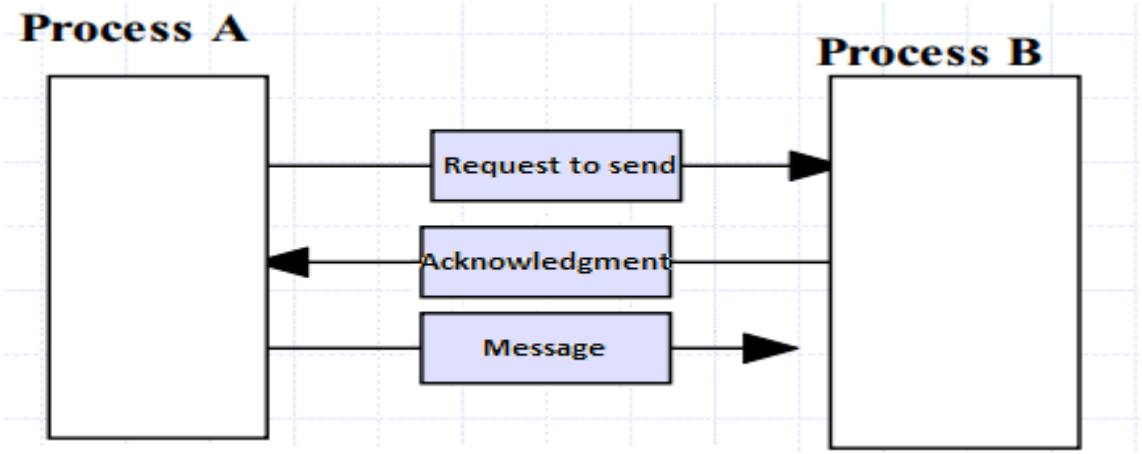
- **Symmetric architecture**
- **Every node can play both the client and server roles (simultaneously or at different times)**

Client/Server Paradigm

- Most widely used paradigm for network applications
- Server plays the role of a service provider which waits passively for the arrival of request
- Client issues specific requests to the server and waits for its response



Synchronous Message Passing

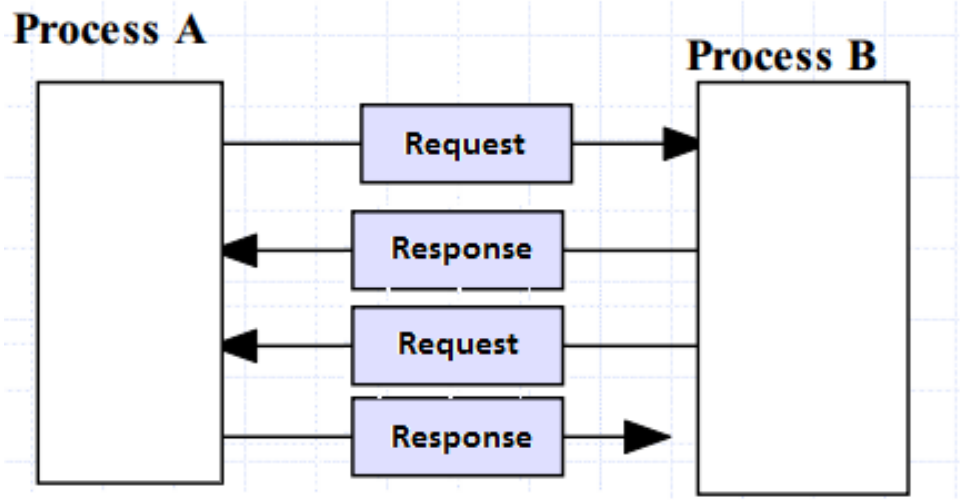


- **Routines return when message transfer completed**
- **Send routine waits until complete message can be accepted by the receiving process before sending the message**
- **Receive routine waits until the message it is expecting arrives**

Asynchronous Message Passing

- **Routines do not wait for actions to complete before returning**
- **Local storage/message queue is required to hold messages**
- **Fast: allow processes to move forward**
- **Less reliable and must be used with care**
- **Message buffer needed between source and destination to hold messages**
- **Send routine wait in case the buffer space is exhausted**

Peer to Peer System Architecture



- In the peer-to-peer paradigm, the participating processes play equal roles, with equivalent capabilities and responsibilities (hence the term “peer”)
- Each participant may issue a request to another participant and receive a response

Peer to Peer System Architecture (Cont.)

- Computer resources and services are directly exchanged between computer systems
- Resources and services include the exchange of information, processing cycles, cache storage, and disk storage for files
- Computers that have traditionally been used solely as clients communicate directly among themselves and can act as both clients and servers, assuming whatever role is most efficient for the network
- The peer-to-peer paradigm is more appropriate for applications such as instant messaging, peer-to-peer file transfers, video conferencing, and collaborative work

Distributed Systems: Issues

Programming Complexity:

- **How the various programs communicate together?**
- **Which data format on the various network nodes?**
- **Need to define (application) protocols**
- **Operations synchronization may lead to delay and slowing down**

Scarce Robustness:

- **Higher chance of errors / faults**

Hard to optimize

- **Lack of global view**

Summary

- **Classic Computing - Pros and Cons**
- **Distributed System**
- **Reasons for Distributed Systems**
- **Examples of Distributed Systems**
- **Search Engine – An example of distributed systems**
- **Advantages of Distributed Systems**
- **Trends in Distributed Systems**
  - **Pervasive Network Technology**
  - **Mobile and Ubiquitous Computing**
  - **Spontaneous Interoperation**

Summary (Cont.)

- **Trends in Distributed Systems (Cont.)**
  - **Distributed Computing as a Utility**
  - **Focus on resource sharing**
- **Forms of Distributed Computing**
  - **Cluster Computing**
  - **Grid Computing**
  - **Cloud Computing**
- **Message Passing Paradigm**
- **Communication Model**
- **Parameters for Measuring Arrival Latency**

Summary (Cont.)

- **Architectures**
  - **Client-Server (C/S) Architecture**
  - **Peer-to-Peer (P2P) Architecture**
- **Client/Server Paradigm**
- **Synchronous Message Passing**
- **Asynchronous Message Passing**
- **Peer to Peer System Architecture**
- **Distributed Systems: Issues**
- **Summary**

Questions?

Class Activity